

Definitions and Short Description

Global Application: The whole project to which all teams participate.

Application: The part of the **global application** this team works on.

Other Application/s: The part of the **global application** other teams work on.

Gem Green: #99d447 (hex)

Financial-Hell Red: #f54c36 (hex)

Electron-Swing Yellow: #ffba3b (hex)

Children: Nested **UI Elements**.

Tag: Reference to another paragraph in the same local scope.

The **application** will have the following **UI Elements** (with additional information and short description):

- **Navbar:**
 - Navigation bar, enabling navigation between pages.
 - Present on the left side of all **UI Pages** except the **Stock Page** and the **Alert Windows**, sharing the viewport.
- **Portfolio:**
 - Main **UI Page**.
 - Showcases basic information about the user's stocks.
- **Stock List:**
 - Showcases a list of all available stocks.
- **History:**
 - Showcases the history of transactions (related to stocks) the user has made.
- **Gem Store:**
 - Showcases a store where the user can buy **GEMS**, the currency of the **application**.
 - Interacts with one **other application** (*Andrada's Team*) for currency to currency exchange.
- **Alert Window/s:**
 - Popup Windows that showcase an alert related to a stock price.
- **Stock Menu:**
 - Showcases details and buy/sell information and abilities related to a stock.
 - Interacts with one **other application** (*Razvan's Team*) for sending alerts on

buy/sell.

The **application** will have the following **NON-UI Elements** (with additional information and description):

- **Global Service:**
 - Will mainly manage:
 - Seamless integration of all pages.
 - Direct communication with **other applications**. (*for encapsulation*)
 - State.
 - Defines the concepts of:
 - **heart-beat**/s: the event of the new random generation of all values for all stocks (*in intervals of equal length of time*).
 - **page change**: the request to change the current page to a different one (specified in the request).
- **Database Wrapper:**
 - Will manage database connection and requests.

Navbar - *Ionut*

A side-bar navigation menu enabling navigation between pages. Present on the left side of all **UI Pages** except the **Stock Page** and the **Alert Window**.

The navbar will have the following **UI Elements**, displayed **vertically** from top to bottom in the following order:

- **Button** displaying the text **"Portfolio"**.
 - Horizontally spanning the whole navbar width. **(FILL)**
 - Vertically spanning the height of the content. **(FIT)**
 - If the **current page** is the one indicated by the button, the background color of the button should be **Gem Green**.
 - Other applied style is not required by the requirement but welcome.
 - When clicked request from **global service** *page change* to the **Portfolio** page.
- **Button** displaying the text **"Stocks"**.
 - Horizontally spanning the whole navbar width. **(FILL)**

- Vertically spanning the height of the content. **(FIT)**
- If the **current page** is the one indicated by the button, the background color of the button should be **Gem Green**.
- Other applied style is not required by the requirement but welcome.
- When clicked request from **global service page** change to the **Stock List** page.
- **Button** displaying the text **"History"**.
 - Horizontally spanning the whole navbar width. **(FILL)**
 - Vertically spanning the height of the content. **(FIT)**
 - If the **current page** is the one indicated by the button, the background color of the button should be **Gem Green**.
 - Other applied style is not required by the requirement but welcome.
 - When clicked request from **global service page** change to the **History** page.
- **Button** displaying the text **"Gem Store"**.
 - Horizontally spanning the whole navbar width. **(FILL)**
 - Vertically spanning the height of the content. **(FIT)**
 - If the **current page** is the one indicated by the button, the background color of the button should be **Gem Green**.
 - Other applied style is not required by the requirement but welcome.
 - When clicked request from **global service page** change to the **Gem Store** page.
- **Container - Wallet:**
 - All applied style is not required by the requirement but welcome.
 - Horizontally spanning the whole navbar width. **(FILL)**
 - Vertically spanning the height of the content. **(FIT)**
 - **Children** are aligned **vertically**.
 - Contains the following **UI Elements**:
 - **Text Label** displaying the text **"Wallet"**
 - All applied style is not required by the requirement but welcome.
 - **Text Label** displaying the text **"Gems: "**, followed by the amount of **gems** the user currently possesses.
 - **Gem Green** colored Text (any shade).
 - Other applied style is not required by the requirement but welcome.

- The text should refresh every time the amount of gems the user possesses changes to reflect the new value.
- **Text Label** displaying the text "**Stocks (as Gems):** " followed by the current '*market value*' of all the stocks possessed by the user.
 - All applied style is not required by the requirement but welcome.
 - The value should be re-calculated every **heart-beat** with the new data provided by the **Global Service** and refreshed in the ui to reflect the new value.
- **Text Label** displaying the user's **username**
- All applied style is not required by the requirement but welcome.

Portfolio - Bianca

The **Portfolio** shows information related to the current value of all stocks possessed by the user, alongside a list of those stocks.

The **Portfolio** contains the following **UI Elements**, displayed **vertically** from top to bottom in the following order:

- **Container - Top Bar:**

- All applied style is not required by the requirement but welcome.
- Horizontally spanning the whole available width. **(FILL)**
- Vertically spanning the height of the content. **(FIT)**
- **Children** are aligned **horizontally** from **left** to **right**.
- **Children** are centered **horizontally** and **vertically**.
- Contains the following **UI Elements**:
 - **Text Label** displaying the text "**Portfolio**".
 - All applied style is not required by the requirement but welcome

- **Container - Information:**

- All applied style is not required by the requirement but welcome.
- Horizontally spanning the whole available width. **(FILL)**
- Vertically spanning the height of the content. **(FIT)**

- **Children** are aligned **horizontally**, from **left** to **right**.
- Contains the following **UI Elements**:
 - **Container - Initial Investment:**
 - **Showcases the value of all stocks possessed by the user at the time they were bought.**
 - All applied style is not required by the requirement but welcome.
 - Horizontally spanning the whole available width. **(FILL)**
 - Vertically spanning the whole available height. **(FILL)** (*parent FIT takes precedence*)
 - **Children** are centered **horizontally** and **vertically**.
 - Contains the following **UI Elements**:
 - **Text Label** displaying the sum of the initial (at the time of buying) price of all **stocks** possessed by the user. (*defined as value*).
 - All applied style is not required by the requirement but welcome.
 - The value should be **regenerated** (or **generated**) from the list of all **Account Stock's** (received from the **global service**) every time the page is reloaded. *There is no need to regenerate while the page is loaded as there is no possibility of buying/selling stocks in the portfolio page.*
 - **Container - Current Value:**
 - **Showcases the current value of all stocks possessed by the user.**
 - All applied style is not required by the requirement but welcome.
 - Horizontally spanning the whole available width. **(FILL)**
 - Vertically spanning the whole available height. **(FILL)** (*parent FIT takes precedence*)
 - **Children** are centered **horizontally** and **vertically**.
 - Contains the following **UI Elements**:
 - **Text Label** displaying the sum of the current price of all

stocks possessed by the user. (*defined as value*).

- All applied style is not required by the requirement but welcome.
- The value should be **regenerated** (or **generated**) from the list of all **Account Stock's** and the list of all **Stocks** (received from the **global service**) every **heart-beat**.

▪ **Container - Gain Percentage:**

- **Showcases the current value of all stocks possessed by the user.**
- All applied style is not required by the requirement but welcome.
- Horizontally spanning the whole available width. (**FILL**)
- Vertically spanning the whole available height. (**FILL**) (*parent FIT takes precedence*)
- **Children** are centered **horizontally** and **vertically**.
- **Children** are aligned **horizontally** from **left** to **right**.

define (*in the local scope*) **perc-value** as the percentage ratio between the sum of the initial (at the time of buying) price of all **stocks** possessed by the user and the sum of the current price of all **stocks** possessed by the user.

- Contains the following **UI Elements**:
 - **Text Label** displaying the absolute value of the **perc-value**.
 - If the **perc-value** is **greater or equal to 0**, the text should have the **Gem Green** color, **Financial-Hell Red** otherwise.
 - Other applied style is not required by the requirement but welcome.
 - The value should be **regenerated** (or **generated**) from the list of all **Account Stock's** and the list of all **Stocks** (received from the **global service**) every **heart-beat**.
 - **Image** displaying an image of an arrow.

- Of **Gem Green** color and pointed **Upwards** if **perc-value** is **greater of equal to 0**, of **Financial Hell Red** and pointed **Downwards** otherwise.
- Format: png , size: 32x32px .
- **Container - User's Stocks:**
 - All applied style is not required by the requirement but welcome.
 - Horizontally spanning the whole available width. **(FILL)**
 - Vertically spanning the whole available height. **(FILL)**
 - **Scrollable**
 - **Children** are aligned **vertically**, from **top** to **bottom**.
 - Is re-populated with all stocks received each heartbeat from the **global service**. Thus, there should be no refresh needed on a per individual stock level.
 - Contains the following **UI Elements**:
 - **Container - Stock: Showcases the symbol, name, current value, percentage of stock value gained/lost since the last heart-beat and an OPEN button.**
 - One instance for each stock possessed by the user.
 - All applied style is not required by the requirement but welcome.
 - Horizontally spanning the whole available width. **(FILL)**
 - Vertically spanning the height of the content. **(FIT)**
 - **Children** are aligned **horizontally** from **left** to **right**.
 - Contains the following **UI Elements**:
 - **Text Label** displaying the **symbol** of the **stock**. (e.g. GOOG)
 - All applied style is not required by the requirement but welcome.
 - **Text Label** displaying the **name** of the **stock**. (e.g. Alphabet Inc)
 - All applied style is not required by the requirement but welcome.
 - **Text Label** displaying the **current value** of the stock.
 - All applied style is not required by the requirement but welcome.
 - This value does not need refreshing as the whole

- parent** container is refreshed every **heart-beat**.
- **Text Label** displaying the **percentage of stock value gained/lost since the last heart-beat**.
 - If the *percentage of stock value gained/lost since the last heart-beat* is higher or equal to 0, the color of the text should be **Gem Green**, the color should be **Financial Hell Red** otherwise.
 - All applied style is not required by the requirement but welcome.
- **Button** displaying the text **“Open”**.
 - **On Click** the button will request from the **global service** *page change* to the **Stock Menu** for the **stock** in cause.

Stock List - Riccardo

The **Stock List** displays a list of all available stocks received from the **global service**. The **Portfolio** contains the following **UI Elements**, displayed **vertically** from **top** to **bottom** in the following order:

- **Container - Controls:**
 - All applied style is not required by the requirement but welcome.
 - Horizontally spanning the whole available width. **(FILL)**
 - Vertically spanning the height of the content. **(FIT)**
 - **Children** are aligned **horizontally** from **left** to **right**.
 - Contains the following **UI Elements**:
 - **Text Input** (*text box*) | defined as **tag: search-input** | displaying the placeholder text **“Stock Symbol or Name”**.
 - All applied style is not required by the requirement but welcome.
 - Horizontally spanning the whole available width. **(FILL)**
 - **Button** displaying the text **“Search!”**.
 - All applied style is not required by the requirement but welcome.

- **On Click** the application shall filter all **stock lists** (*favorite and other | see below*), only keeping entries that contain the **case-insensitive** text inserted in the text input (*tag: search-input*). If the input (*tag: search-input*) is empty, **all available stocks** shall be listed. The change shall be immediately reflected in the two lists.
- **Text Label** displaying the text **"Sort By"**.
 - All applied style is not required by the requirement but welcome.
- **Drop-Down Menu** | defined as **tag: order-input* | a dropdown menu for sorting options.
 - All applied style is not required by the requirement but welcome.
 - Displaying the following options:
 - **Name**: stock name. - *default*
 - **Price**: stock price.
 - **Percentage** stock gain/loss percentage ratio between the last 2 **heart-beats**.
 - When an option is selected in the menu, the change shall be **instantly reflected** in all **stock lists** (*favorite and other | see below*).
- **Container - Lists**:
 - All applied style is not required by the requirement but welcome.
 - Horizontally spanning the whole available width. **(FILL)**
 - Vertically spanning the whole available height. **(FILL)**
 - **Scrollable**
 - **Children** are aligned **vertically** from **top** to **bottom**.
 - Contains the following **UI Elements**:
 - **Container - Favorite**: a list displaying the favorite stocks.
 - **Background Color**: **Electro-Swing Yellow** // *its 2 AM :C , i.m out of names*
 - Other applied style is not required by the requirement but welcome.
 - Horizontally spanning the whole available width. **(FILL)**
 - Vertically spanning the height of the content. **(FIT)**
 - **Children** are aligned **vertically** from **top** to **bottom**.

- Contains the following **UI Elements**:
 - All **stocks** (*tag: stock*) received from the **global service** that are found the favorite list received from the **global service** and pass the search requirements (*tag: search-input*) ordered by requirements (*tag: order-input*).
 - The list is regenerated each **heart-beat** or if requested via search, filter (*tag: search-input*, *tag: order-input*) or by other events.
- **Container - NoN-Favorite**: a list displaying the remaining stocks (that are not favorite).
 - All applied style is not required by the requirement but welcome.
 - Horizontally spanning the whole available width. **(FILL)**
 - Vertically spanning the height of the content. **(FIT)**
 - **Children** are aligned **vertically** from **top** to **bottom**.
 - Contains the following **UI Elements**:
 - All **stocks** (*tag: stock*) received from the **global service** that are not found the favorite list received from the **global service** and pass the search requirements (*tag: search-input*) ordered by requirements (*tag: order-input*).
 - The list is regenerated each **heart-beat** or if requested via search, filter (*tag: search-input*, *tag: order-input*) or by other events.

Define **tag: stock** as:

- **Container - Stock**: *generated with a stock model (symbol, name, price, percentage)*
 - All applied style is not required by the requirement but welcome.
 - Horizontally spanning the whole available width. **(FILL)**
 - Vertically spanning the height of the content. **(FIT)**
 - **Children** are aligned **horizontally** from **left** to **right**.
 - Contains the following **UI Elements**:
 - **Text Label** displaying the **Symbol** of the stock.
 - All applied style is not required by the requirement but welcome.
 - **Text Label** displaying the **Name** of the stock.
 - All applied style is not required by the requirement but

welcome.

- **** Text Label**** displaying the **Price** of the stock followed by the text "**Gems**".
 - All applied style is not required by the requirement but welcome.
- **Text Label** displaying the **Percentage** of the stock.
 - The percentage ratio between the stock price of the last two **heart-beats** (*defined as value*).
 - If the value is **greater or equal to 0**, the color of the text should be **Gem Green**, otherwise, the color should be **Financial-Hell Red**.
 - Other applied style is not required by the requirement but welcome.
- **Image** displaying:
 - format: png , size: 32x32px
 - If the stock is listed in the favorite list provided by the **global service**:
 - Displaying a **Filled Star Shape** in the color **Electro-Swing Yellow**.
 - **On click** the stock is removed from the **favorite list** and both list containers described above are regenerated. The new **favorite list** is also forwarded to the **global service** to be processed and saved.
 - Otherwise:
 - Displaying a **Star Outline** in the color **Electro-Swing Yellow**.
 - **On click** the stock is added to the **favorite list** and both list containers described above are regenerated. The new **favorite list** is also forwarded to the **global service** to be processed and saved.

History - *Denis*

An UI page that displays the transaction history of the user related to stock trading.

Design:

The History page will have the following **UI Elements**, displayed vertically from top to bottom:

- **Container - Top Bar:**

- Displays the page title and ensures proper alignment with other UI elements.
- Horizontally spanning the whole available width. **(FILL)**
- Vertically spanning the height of the content. **(FIT)**
- Children are aligned horizontally from left to right.
- Children are centered both horizontally and vertically.
- Contains the following **UI elements**:
 - **Text Label** displaying the text **"History"**.
 - All applied style is not required by the requirement but welcome.
 - **Button** displaying the text **"Save As"**.
 - On Click, it prompts the user to save the transaction history as a file (e.g., CSV or PDF).
 - Horizontally spanning the height of the content. **(FIT)**

- **Container - Filters:**

- Allows filtering transactions based on stock name, type, and date range.
- Horizontally spanning the whole available width. **(FILL)**
- Vertically spanning the height of the content. **(FIT)**
- Children are aligned horizontally, from left to right.
- Contains the following **UI elements**:
 - **Dropdown Menu** for selecting **Stock Name**.
 - **Dropdown Menu** for selecting **Transaction Type (BUY / SELL / ALL)**.
 - **Date Picker** for selecting a **Start Date - End Date Range**.
 - **Button** displaying the text **"Apply Filters"**.
 - Horizontally spanning the height of the content. **(FIT)**
 - On Click, it applies the selected filters to the transaction list.

- **Container - Transactions List:**

- Displays a list of all stock transactions.
- Horizontally spanning the whole available width. **(FILL)**

- Vertically spanning the whole available height. **(FILL)**
- Children are aligned vertically, from top to bottom.
- The list is re-populated every **heart-beat** based on new transactions received from the **Global Service**.
- Contains the following **UI elements**:
 - **Container - Transaction Row**:
 - Displays transaction details in a row format.
 - Horizontally spanning the whole available width. **(FILL)**
 - Vertically spanning the height of the content. **(FIT)**
 - Children are aligned horizontally, from left to right.
 - Contains the following **UI elements**:
 - **Text Label** displaying the **Stock Symbol** (e.g., GOOG).
 - **Text Label** displaying the **Stock Name** (e.g., Alphabet Inc).
 - **Text Label** displaying the **Transaction Type (BUY / SELL)**.
 - BUY transactions should be displayed in **Gem Green**.
 - SELL transactions should be displayed in **Financial-Hell Red**.
 - **Text Label** displaying the **Amount** of stocks transacted.
 - **Text Label** displaying the **Price per Stock**.
 - **Text Label** displaying the **Total Value** (Amount × Price per Stock).
 - **Text Label** displaying the **Timestamp** of the transaction.
- **Container - Sorting Options**:
 - Allows sorting transactions based on different criteria.
 - Horizontally spanning the whole available width. **(FILL)**
 - Vertically spanning the height of the content. **(FIT)**
 - Children are aligned horizontally, from left to right.
 - Contains the following **UI elements**:
 - All applied style is not required by the requirement but welcome.
 - Displaying the following options:
 - **Dropdown Menu** for selecting sorting criteria:
 - **Date (Newest First / Oldest First)**
 - **Stock Name (A-Z / Z-A)**

- **Transaction Value (Highest / Lowest)**
- When an option is selected in the menu, the change shall be instantly reflected in all stock lists

Global Service - *losua*

Stock:

- name: text
- symbol: text
- price: number
- percentage: number (the percentage ratio between the last two **heart-beats** prices)

Account Stock:

- name: text
- symbol: text
- cumulative bought price: number (the sum of all the buy price of a bought stock type)
- amount: number

History Stock:

- type: BUY | SELL
- name: text
- symbol: text
- price: number
- amount: number
- date: date

The **Global Service** ensures seamless integration of parts of the **application**:

- **State Management**
 - **The application** shall manage the state of all data in the application. (*Load*,

Save, Update)

- **The application** shall **load** information about the user,
 - **The application** shall **store** information about the user:
 - **UserID:** *|String|* The ID of the user (request from **other application** - *Andrada's Team*).
 - **Username:** *|String|* The Username of the user (request from **other application** - *Andrada's Team*).
 - **Gems** *|Integer|*:
 - First initialization value: **0**
 - **List of Account Stocks:** A list with all the stocks that the user possesses.
 - **List of History Stocks:** A list of the transaction history of the user.
 - **Favorite List:** A list of the names of the stocks the user has added to **favorites**.
 - **Alerts:** A list of the names of the stocks the user has added to **alerts** along with their **alert bound**. (*upper and lower bound*)
 - If the information is not stored in the **database** generate it with default values: (**empty**, **0**, **""**)
 - On each change to the information **the application** shall store it to the **database**.
- **Live Stock Value:**
 - At startup **the application** contains the following **stocks**:
 - BKNG: Booking Holdings Inc
 - ORLY: O'Reilly Automotive Inc
 - MSTR: MicroStrategy
 - MELI: MercadoLibre Inc
 - KLAC: KLA Corp
 - NFLX: Netflix Inc
 - AXON: Axon Enterprise Inc
 - CRWD: CrowdStrike Holdings Inc
 - APP: Applovin Corp
 - ASML: ASML Holding NV
 - SNPS: Synopsys Inc

- META: Meta Platforms Inc
- INTU: Intuit Inc
- ADBE: Adobe Inc.
- COST: Costco Wholesale Corp
- TEAM: Atlassian Corp
- MSFT: Microsoft Corp
- TSLA: Tesla Inc
- IDXX: IDEXX Laboratories Inc
- ZS: Zscaler Inc
- ISRG: Intuitive Surgical Inc
- ADSK: Autodesk Inc
- WDAY: Workday Inc
- PLTR: Palantir Technologies Inc
- CEG: Constellation Energy Corp
- ROP: Roper Technologies Inc
- CHTR: Charter Communications Inc
- NVDA: NVIDIA Corp
- PANW: Palo Alto Networks Inc
- MU: Micron Technology Inc
- Price is **generated randomly** between (100 and 500) at **startup**.
- A price history (*not transaction history*) of 900 **heart-beats** is generated on startup for each stock. These will be used by the **Stock Menu** to have a broader history range in the chart. (**hb = heart-beats**, 30 - standard length of the chart, $30 * 1 \text{ hb} = 30 \text{ hb}$, $30 * 10 \text{ hb} = 300 \text{ hb}$, $30 * 30 \text{ hb} = 900 \text{ hb}$)
- On each **heart-beat** the history of prices of all stocks is shifted by one and a new **HEAD** is generated using the following formula:

```
x = rand(-5, 5)
max(5, old_head_price + x)
```

- If the value **becomes too low** it is **capped** at 5.
- **Page Change request**
 - The **current page** (*or the navbar*) can request a *page change* to a different page (by id). The old page id should be saved in case the **new page** allows the user to **go back** (*stock page*).
- **Provide Required Data:**

- Provide required data to:
 - **Navbar:**
 - Username
 - Gems
 - List of Account Stocks
 - **Portfolio:**
 - List of Account Stocks
 - List of Stocks
 - **Stock List:**
 - List of Account Stocks
 - List of Stocks
 - List of Favorite Stock Names
 - **History:**
 - List of History Stocks
 - **Gem Store:**
 - Gems
 - **On request:**
 - List of Banking Accounts (*from other application*)
 - **Alert Window:**
 - List of Stocks
 - List of Alerts (*stock names, bounds*)
 - **Stock Menu:**
 - List of Account Stocks
 - List of List of Stocks (*List of stocks with history*)
- Communication with **Other Applications:**
 - Andrada's team:
 - request available accounts (list) (for buying gems).
 - returns a list of all available accounts.
 - request subtraction of funds from X account (where X is provided by the application via the user). (*see store page*)
 - The request must return the status of the procedure, successful, failed: not enough funds, failed: internal error.
 - request addition of funds to account X (where X is provided by the application via the user). (*see store page*)
 - request should return the status of the procedure, successful, failed.

- Razvan's team:
 - send notifications each time a user buys or sells a stock.
 - notification should contain:
 - action: buy/sell
 - stock symbol: text
 - stock name: text
 - buy/sell price: number

Alert System - Norbert

The **Alert System** allows users to define and manage alerts for stock price changes. The **Alert System** includes the following **UI Elements**, displayed and interacted with as follows:

- **Alert Window**

- A popup modal where users can configure alerts for a stock's price.
- **Children** are aligned **vertically**, from **top** to **bottom**.
- Contains the following **UI Elements**:
 - **Text Label** displaying the text **"Up To"**.
 - All applied style is not required by the requirement but welcome.
 - **Text Input** (tag: up-to-inpit)
 - All applied style is not required by the requirement but welcome.
 - **Text Label** displaying the text **"Down To"**.
 - All applied style is not required by the requirement but welcome.
 - **Text Input** (tag: down-to-inpit)
 - All applied style is not required by the requirement but welcome.
 - **Button** displays the text "Save Alert"
 - All applied style is not required by the requirement but welcome.

- **On Click** validate the input:
 - If the input is not valid ($0 < \text{lower bound} < \text{upper bound}$, numbers) show error popup.
 - Otherwise save alert in **Global Service**
- **Button** displays the text “Cancel”
 - All applied style is not required by the requirement but welcome.
 - **On Click** revert back to Stock Menu.
- **Notification System**
 - Displays notifications when an alert's condition is met.
 - **Uses WinUI 3 notifications** for desktop alerts.
 - Contains the following **UI Elements**:
 - **Text Label** displaying the **Stock Symbol** and **Name**.
 - **Text Label** indicating whether the alert was “**Up To**” or “**Down To**”.
 - **Text Label** displaying the **current stock price** at the time of the trigger.

Alert Behavior & Logic

Alert Creation

1. The user clicks the **Alert Button** on a stock item.
2. The **Alert Window** opens, allowing the user to enter “**Up To**” and “**Down To**” prices.
3. Clicking **Save Alert**:
 - Validates input (ensuring values are valid numbers in range).
 - Creates a new **Alert** associated with the stock.
 - Initializes `IsActive = true` and `HasBeenTriggered = false`.
 - Stores the alert.

Alert Monitoring

- The **Global Service** periodically updates stock prices.
- For each **active alert**, it checks if:

- The current stock price **exceeds** the **AlertUpToPrice**.
- The current stock price **falls below** the **AlertDownToPrice**.
- If an alert is triggered:
 - HasBeenTriggered is set to `true`.
 - A notification is displayed.
 - The alert remains active unless manually **deleted** or **deactivated**.
 -

Stock Menu - *Rafa* ()

The **Stock Menu** shows information related to a selected Stock.

The **Stock Menu** contains the following **UI Elements**, displayed **vertically** from **top** to **bottom** in the following order:

- **Container - Top Bar:**
 - All applied style is not required by the requirement but welcome.
 - Horizontally spanning the whole available width. **(FILL)**
 - Vertically spanning the height of the content. **(FIT)**
 - **Children** are aligned **horizontally** from **left** to **right**.
 - Contains the following **UI Elements**:
 - **Text Label** displaying the **symbol of the stock**.
 - All applied style is not required by the requirement but welcome.
 - **Text Label** displaying the **name of the stock**.
 - All applied style is not required by the requirement but welcome.
 - **Text Label** displaying the text **"Gems: "** and **the amount of gems the user possesses**.
 - All applied style is not required by the requirement but welcome.
 - After each buy/sell the value will be refreshed.
 - **Button** displaying the text **"Exit"**
 - All applied style is not required by the requirement but welcome.

- **On Click** reverts back to the page that launched the menu. (via the **global service**)
- **Chart:**
 - All applied style is not required by the requirement but welcome.
 - Horizontally spanning the whole available width. **(FILL)**
 - Vertically spanning the whole available height. **(FILL)**
 - **Using the livechart2 winui3 library.**
 - Update the Graph on each **heart-beat** to reflect the new values added to the **stock value history**.
- **Container - Bottom Bar:**
 - All applied style is not required by the requirement but welcome.
 - Horizontally spanning the whole available width. **(FILL)**
 - Vertically spanning the height of the content. **(FIT)**
 - **Children** are aligned **horizontally** from **left** to **right**.
 - Contains the following **UI Elements**:
 - **Text Label** displaying the text "**Current Value:** " and the **current value** of the stock.
 - All applied style is not required by the requirement but welcome.
 - the value will be updated on each **heart-beat**.
 - **Text Label** displaying the text "**You own:** " and the **quantity** of the stock **owned** by the user.
 - All applied style is not required by the requirement but welcome.
 - The value shall refresh on each buy/sell to reflect the new value.
 - **Text Label** displaying the text "**Select Quantity:** ".
 - All applied style is not required by the requirement but welcome.
 - **Number Input:**
 - All applied style is not required by the requirement but welcome.
 - Selecting the quantity of stocks to buy/sell.
 - Default value: 0
 - **Button** displaying the text "**Buy!**":
 - All applied style is not required by the requirement but

welcome.

- **On Click** validate if the user can afford the quantity * price:
 - If user can not afford: Show error popup.
 - Otherwise request **buying stock** from **global service** and refresh all the elements specified above.
- **Button** displaying the text **"Sell!"**:
 - All applied style is not required by the requirement but welcome.
 - **On Click** validate if the user has the quantity of stock selected:
 - If user does not have enough stocks: Show error popup.
 - Otherwise request **selling stock** from **global service** and refresh all the elements specified above.
- **Button** displaying text **"Set Allert"**:
 - All applied style is not required by the requirement but welcome.
 - **On Click** request **global service** to open **Alert Window** for the stock.
- **Toggle Button** displaying text **"Favorite"**:
 - All applied style is not required by the requirement but welcome.
 - **On Click** if:
 - Stock is **not on the favorite list**, request from **global service** to add stock to favorite list.
 - Otherwise, request from **global service** to remove stock from favorite list.

Gem Store - Ana ()

The **Gem Store** allows the user to buy/sell gems for normal currency.

The **Gem Store** contains the following **UI Elements**, displayed **vertically** from **top** to **bottom** in the following order:

- **Container - All Deals:**
 - All applied style is not required by the requirement but welcome.

- Horizontally spanning the whole available width. **(FILL)**
- Vertically spanning the whole available height. **(FILL)**
- **Children** are aligned **vertically** from **top** to **bottom**.
- **Scrollable**
- Contains the following **UI Elements**:
 - `gem_deal("LEGENDARY DEALLLL!!!", 4999, 100.0)`
 - `gem_deal("MYTHIC DEAL!!!", 3999, 90.0)`
 - `gem_deal("INSANE DEALLLL!!!", 3499, 85.0)`
 - `gem_deal("GIGA DEAL!!!", 3249, 82.0)`
 - `gem_deal("WOW DEAL!!!", 3000, 80.0)`
 - `gem_deal("YAY DEAL!!!", 2500, 50.0)`
 - `gem_deal("YUPY DEAL!!!", 2000, 49.0)`
 - `gem_deal("HELL NAH DEAL!!!", 1999, 48)`
 - `gem_deal("BAD DEAL!!!", 1000, 45.0)`
 - `gem_deal("MEGA BAD DEAL!!!", 500, 40.0)`
 - `gem_deal("LEGENDARY BAD DEAL!!!", 1, 35.0)`
- **Container - Sell:**
 - All applied style is not required by the requirement but welcome.
 - Horizontally spanning the whole available width. **(FILL)**
 - Vertically spanning the height of the content. **(FIT)**
 - **Children** are aligned **horizontally** from **left** to **right**.
 - Contains the following **UI Elements**:
 - **Container - Title-Value-Price:**
 - All applied style is not required by the requirement but welcome.
 - Horizontally spanning the width of the content. **(FIT)**
 - Vertically spanning the whole available height. **(FILL)**
 - **Children** are aligned **vertically** from **top** to **bottom**.
 - Contains the following **UI Elements**:
 - **Text Label:** displays the text **"100 Gem"**.
 - All applied style is not required by the requirement but welcome.
 - **Text Label:** displays the text **"1 €"**.
 - All applied style is not required by the requirement but welcome.
 - **Number Input** |define tag: *quantity-inp*| quantity to sell

- All applied style is not required by the requirement but welcome.
- Default Value: **0**
- **Button** displaying the text **"Sell!"**
 - All applied style is not required by the requirement but welcome.
 - **On Click** validate if value of (*tag: quantity-inp*) is > 0 and if user has enough funds:
 - If valid the **accout selector window pops up**.
 - Otherwise: Display popup Error.

Below I've defined a set of macros for easy repeatability

Define MACRO: tag gem_deal(title: string, gem_value: integer, euro_price: float);

• **Container - Deal:**

- All applied style is not required by the requirement but welcome.
- Horizontally spanning the whole available width. **(FILL)**
- Vertically spanning the height of the content. **(FIT)**
- **Children** are aligned **horizontally** from **left** to **right**.
- Contains the following **UI Elements**:
 - **Container - Title-Value-Price:**
 - All applied style is not required by the requirement but welcome.
 - Horizontally spanning the width of the content. **(FIT)**
 - Vertically spanning the whole available height. **(FILL)**
 - **Children** are aligned **vertically** from **top** to **bottom**.
 - Contains the following **UI Elements**:
 - **Text Label:** displays the **title**.
 - All applied style is not required by the requirement but welcome.
 - **Text Label:** displays the **gem_value** followed by the text **"Gems"**.
 - All applied style is not required by the requirement but welcome.
 - **Text Label:** displays the **euro_price** followed by the text **"€"**.
 - All applied style is not required by the requirement

but welcome.

- **Button** displaying the text **“Buy!”**.
 - All applied style is not required by the requirement but welcome.
 - **On Click** the **account selector window pops up**.

Account Selection Window - *Iosua*

The **Account Selection Window** allows the user to select which account to use for transactions.

The **Account Selection Window** contains the following **UI Elements**, displayed **vertically** from **top** to **bottom** in the following order:

- **Container - Main:**

- All applied style is not required by the requirement but welcome.
- Horizontally spanning the whole available width. **(FILL)**
- Vertically spanning the whole available height. **(FILL)**
- **Children** are aligned **vertically** from **top** to **bottom**.
- **Scrollable**
- Contains the following **UI Elements**:
 - **Button** displaying the text **“Cancel”**.
 - All applied style is not required by the requirement but welcome.
 - **On Click** the popup responds to the **global service** with **null**

- **Container - Account:**

- An instance for each account in the **account list** received from the **global service**. `*(account-name: text , account-balance: float )`
 - All applied style is not required by the requirement but welcome.
 - Horizontally spanning the whole available width. **(FILL)**
 - Vertically spanning the height of the content. **(FIT)**
 - **Children** are aligned **horizontally** from **left** to **right**.
 - Contains the following **UI Elements**:

- **Text Label:** displaying the **account-name**.
 - All applied style is not required by the requirement but welcome.
- **Text Label:** displaying the **account-balance** and the text "€".
 - All applied style is not required by the requirement but welcome.
- **Button** displaying the text **"Choose"**.
 - All applied style is not required by the requirement but welcome.
 - **On Click** the popup responds to the **global service** with the **account-name**.

Database Wrapper - *lonut*

The **Database Wrapper** allows **the application** to store and load data **persistently**.

- The database wrapper exposes an api to the **Global Service** for managing data.